

Efficient LLM Serving with KV Cache, Speculative Decoding, and Quantization: A Comprehensive Survey

Abstract

The deployment of Large Language Models (LLMs) at massive scale has emerged as a premier challenge in modern artificial intelligence and systems engineering. While the parameter counts and reasoning capabilities of foundation models have scaled exponentially, the underlying hardware capacity to serve these models—specifically high-bandwidth memory (HBM) capacity and memory bandwidth—has not kept pace. Consequently, the autoregressive text generation process is fundamentally bottlenecked by memory bandwidth rather than computational throughput. This comprehensive survey investigates the tripartite pillars of modern LLM serving efficiency: Key-Value (KV) cache optimization, speculative decoding, and low-precision quantization. By systematically reviewing recent algorithmic and system-level advancements, this analysis demonstrates how the field is transitioning from isolated heuristics to mathematically grounded, hardware-aware co-designs. The investigation covers the evolution of dynamic memory management strategies, including PagedAttention and RadixAttention, alongside structured cache eviction and novel architectural paradigms such as Multi-head Latent Attention (MLA). Furthermore, it explores how speculative decoding breaks the sequential dependency of autoregressive generation by speculating in the feature space and utilizing dynamic verification trees. Finally, the survey details post-training quantization (PTQ) techniques that compress weights, activations, and KV states down to sub-4-bit regimes, alongside the emerging algorithmic-system co-designs that enable these theoretical memory savings to yield empirical wall-clock speedups.

Introduction

The unprecedented capabilities of generative Large Language Models (LLMs) have catalyzed their integration into a vast array of applications, spanning from conversational agents and programming assistants to complex multi-step reasoning engines [3, 50, 188]. However, the autoregressive nature of Transformer-based architectures dictates that text is generated sequentially, token by token. For each generated token, the model must access all of its weights and the historical states of all preceding tokens in the sequence. Because modern Graphics Processing Units (GPUs) and specialized accelerators process arithmetic operations significantly faster than they can read data from High-Bandwidth Memory (HBM), LLM decoding operates in a severely memory-bandwidth-bound regime [18, 121, 171].

To alleviate redundant computations during the decoding phase, inference systems employ the Key-Value (KV) cache, a mechanism that stores the intermediate key and value tensors of historical tokens. While caching avoids recomputing past states, its memory footprint scales linearly with the sequence length, the batch size, and the model's hidden dimension [7, 9, 138].

For massive models processing contexts extending into hundreds of thousands or millions of tokens, the KV cache can consume terabytes of memory. This footprint frequently dwarfs the size of the model weights themselves, forcing systems into serving single-digit batch sizes and severely degrading overall throughput [7, 71, 188].

Addressing the memory wall requires navigating the intricate dynamics of the CAP principle for LLM serving, which posits an inherent trilemma between Cost-efficiency (C), serving Accuracy (A), and serving Performance/Context length (P) [4]. Optimizing along any single axis typically necessitates concessions in the others. Furthermore, cloud-scale deployments face strict power constraints; systems like POLCA characterize the power consumption patterns of LLM inference clusters and dynamically apply GPU frequency locking and power capping, while frameworks like PerLLM leverage edge-cloud collaboration to optimize service scheduling under stringent energy budgets [3]. Concurrently, evaluating serving performance requires user-centric metrics. Systems such as Andes formulate Quality of Experience (QoE) metrics based on expected Token Delivery Timelines (TDT) and reading speeds, utilizing dynamic priority-based preemptive schedulers to optimize hardware allocation without generating text faster than the user can consume it [3]. Similarly, FlexLLM enables the co-location of memory-bound LLM inference and compute-bound Parameter-Efficient Fine-Tuning (PEFT) within the same iteration, breaking down fine-tuning into token-level computations to maximize GPU tensor core utilization [3]. Early optimization efforts in the LLM serving space relied heavily on generic heuristics. However, LLM inference serving has outgrown these approaches and now demands mathematical optimization and algorithmic foundations [1]. This survey categorizes and critically analyzes the three primary avenues of system-level and algorithmic enhancements that improve serving performance without fundamentally altering the core pre-training objectives. First, we examine KV Cache Optimization, encompassing memory management, eviction, and architectural restructuring. Second, we dissect Speculative Decoding, focusing on generation-refinement frameworks that introduce intra-request parallelism. Third, we evaluate Quantization strategies that utilize low-precision data representations to compress weights, activations, and KV states.

Methods

The landscape of LLM inference optimization is highly multidimensional. In this section, we organize the research area into coherent method families, detailing the theoretical mechanisms, hardware implications, and empirical performance of each approach.

Dynamic Memory Management and System-Level Scheduling

Early LLM serving systems suffered from severe internal and external memory fragmentation. Because sequence lengths in natural language generation are inherently unpredictable, early systems pre-allocated contiguous blocks of maximum-length memory for the KV cache. This static allocation led to vast amounts of stranded, unusable GPU memory, crippling the maximum batch size [49, 54].

To resolve this inefficiency, PagedAttention was introduced as a foundational mechanism inspired by operating system virtual memory and paging [49, 50, 54]. PagedAttention divides the KV cache into fixed-size blocks (pages) that store the attention keys and values for a discrete

number of tokens. These blocks are mapped via a block table from a contiguous logical space to non-contiguous physical GPU memory [50, 54]. This design essentially eliminates external fragmentation and restricts internal fragmentation to the final, partially filled block of a sequence [50, 54]. By facilitating block-level memory sharing, PagedAttention also enables Copy-on-Write (CoW) sharing for complex sampling techniques like beam search and parallel decoding, drastically reducing the memory footprint for multi-sequence requests [51, 54].

Building upon the PagedAttention paradigm, the SGLang framework introduced RadixAttention to automatically reuse KV caches across distinct requests [55, 56, 58, 60]. Recognizing that many inference workloads—such as agentic loops, few-shot prompting, and multi-turn conversations—share extensive prompt prefixes, RadixAttention organizes the KV cache into a server-wide radix tree [57, 58]. When a new request arrives, the scheduler performs prefix matching against the radix tree to seamlessly reuse existing physical blocks, drastically reducing Time-to-First-Token (TTFT) and prefill memory consumption [56, 59]. The eviction policy operates as a least-recently-used (LRU) mechanism over tree branches, prioritizing the retention of high-fanout branches that represent frequently shared prefixes [57].

Despite its ubiquity, PagedAttention's non-contiguous physical memory layout introduces overhead, particularly when dispatching custom CUDA kernels and performing pointer arithmetic. To mitigate this, vAttention leverages low-level CUDA Virtual Memory Management (VMM) APIs to allocate memory that is physically fragmented but virtually contiguous [121, 124, 125]. By preserving virtual contiguity, vAttention allows off-the-shelf, highly optimized attention kernels (such as standard FlashAttention) to operate without the complex indexing overhead required by PagedAttention [121, 124]. vAttention achieves this by overlapping memory allocation with compute, opportunistically allocating pages ahead of time, and deferring memory reclamation, resulting in up to 1.23x throughput improvements over paged kernels [121, 124, 126].

For deployments requiring massive state retention that exceeds GPU HBM, hierarchical storage systems have emerged. SGLANG-LSM extends cache management by utilizing Log-Structured Merge-trees (LSM-trees) to manage KV caches across disk and CPU tiers, solving the scalability bottlenecks of file-per-object layouts through key-value separation and adaptive controllers [60]. Similarly, OOMB incorporates a paged memory manager that spans both KV cache and gradients during training and inference, utilizing custom Triton kernels with in-place gradient accumulation to bypass the PyTorch autograd engine, thus preventing out-of-memory errors in infinite-context regimes [53].

System/Algorithm	Primary Mechanism	Memory Fragmentation Resolution	Hardware/Software Interface
PagedAttention [50, 54]	OS-inspired paging for KV tensors	Block tables map logical contiguous space to physical non-contiguous	Custom CUDA kernels required for block fetching

		memory	
RadixAttention [58, 60]	Radix-tree based prefix caching	LRU eviction on tree branches; exact prefix matching	Integrated into SGLang scheduler; transparent to user
vAttention [121, 124]	CUDA VMM API allocation	Physical fragmentation allowed; strict virtual contiguity maintained	Supports standard FlashAttention kernels directly
SGLANG-LSM [60]	LSM-tree database storage	Key-value separation for massive tensors across disk/RAM/GPU	Adaptive controller manages tier promotion/demotion

Key-Value Cache Eviction and Sparsification

While dynamic memory management optimizes *how* the cache is stored, token eviction dictates *what* is stored by bounding the absolute size of the cache. Because the attention matrix in LLMs is highly sparse—meaning a small subset of tokens receives the vast majority of attention weights—retaining the full sequence is computationally redundant [129].

Structured eviction methods discard tokens entirely based on specific scoring mechanisms. The Heavy-Hitter Oracle (H2O) framework leverages the persistence of importance hypothesis, tracking cumulative attention scores during generation [61, 62, 64]. H2O evicts "non-heavy-hitter" tokens that consistently receive low attention, safeguarding a strict budget of high-value tokens. Advanced iterations of this concept utilize a semantics-aware memory hierarchy, sorting tokens into four tiers: T0 (HBM) for protected heavy-hitters, T1 (DDR) for medium-importance tokens prefetched to the GPU, T2 (Compressed) for low-importance tokens stored in reduced precision, and T3 (Evicted) for tokens permanently discarded [61].

Conversely, Token Omission Via Attention (TOVA) utilizes the strict next-token distribution to omit the least-attended tokens once a fixed budget is met, treating Transformers as Multi-State RNNs [127, 131, 168].

However, simplistic heuristic eviction can suffer from "selective amnesia" in multi-instruction tasks. Standard eviction policies often exhibit a bias that inadvertently targets system prompts or specific instructions, leading to critical capability failures [130, 132]. To address this, dynamic and context-aware methods have been developed. VaSE (Value-aware Stochastic Eviction) identifies that a small fraction of value states possess abnormally large magnitudes; evicting

these causes catastrophic failure where models enter repetitive reasoning loops [25]. VaSE protects these large-magnitude value states and introduces stochasticity to increase cache diversity [25]. The VECTOR augmentation framework introduces a three-way token routing mechanism (retention, approximation, and eviction), combining importance signals with reconstructability signals to recover useful value information that would otherwise be irreversibly lost under binary eviction [26].

To provide mathematical rigor to eviction, DefensiveKV formulates critical entry identification through the lens of attention output perturbation. This theoretical framework proves that relying solely on attention weights is insufficient; the value states and pretrained parameter matrices heavily influence the worst-case output perturbation [105, 106]. By optimizing eviction to minimize this perturbation bound, DefensiveKV drastically reduces the compression loss of heuristic methods across long-context benchmarks [106]. Similarly, the Information Bottleneck principle has been leveraged to characterize the effective information capacity of retained KV subsets, optimizing eviction through closed-form mutual information objectives under linear-Gaussian surrogates [27, 108].

Furthermore, eviction strategies are shifting from global policies to layer-wise heterogeneous allocations. Frameworks like PolyKV recognize that different transformer layers exhibit vastly different sensitivities to cache reduction. PolyKV routes each layer to a suitable KV compression policy based on offline calibration signals, assigning non-uniform budgets under a fixed total constraint to recover substantial performance gaps compared to single-policy baselines [128]. Beyond token-level eviction, MixedDimKV allocates dimensions to tokens at a granular level, treating eviction as a form of dimensionality reduction that operates dynamically across both tokens and attention heads [29]. Reinforcement learning has also been applied; KVP (KV Policy) trains lightweight per-head RL agents on generation traces to predict the future utility of tokens, completely detaching eviction from historical attention proxies and generating adaptive policies that generalize beyond their training distribution [28].

Structural and Architectural Attention Optimization

Beyond post-training pruning, fundamental alterations to the Transformer architecture have been developed to intrinsically lower KV cache demands and improve computational intensity during inference.

Grouped-Query Attention (GQA) reduces the number of KV heads by sharing them across multiple query heads, lowering the memory footprint compared to standard Multi-Head Attention (MHA) [188, 199]. However, GQA's fixed structural compression can constrain the model's representational capacity and reasoning fidelity.

A profound paradigm shift occurred with the introduction of Multi-Head Latent Attention (MLA), pioneered by the DeepSeek-V2 and DeepSeek-V3 architectures [188, 193]. MLA utilizes low-rank matrices to compress the traditional Key and Value states into a singular, highly

compact latent vector ($d_c \ll d_h n_h$). During autoregressive generation, this compressed latent vector is cached. For attention computation, the latent vector is up-projected through weight matrices to recover the necessary keys and values [188, 196]. A critical innovation within MLA is the decoupled Rotary Position Embedding (RoPE); because position embeddings

cannot be easily compressed via low-rank projections, MLA computes and caches them separately in a lightweight auxiliary tensor [188, 196]. This architecture enables highly efficient generation with a KV cache footprint that is 93.3% smaller than standard MHA, without sacrificing the expressive power of the network [188, 199].

The efficiency of MLA has prompted the development of translation frameworks, such as TransMLA and MHA2MLA, which convert existing GQA-based models (e.g., Llama-3, Qwen) into MLA architectures post-training [199, 200, 203]. These frameworks utilize modality-decoupled low-rank approximations and partial-RoPE masking strategies to align the attention spaces. Following this conversion, highly data-efficient parameter-efficient fine-tuning (PEFT) is applied—requiring only a fraction of a percent of the original training data—to recover baseline performance while retaining the massive inference acceleration of the latent cache [200, 203]. LatentLLM further extends this concept by applying global attention-aware joint tensor decomposition across query, key, value, and output projections simultaneously [202]. Another vector of architectural innovation exploits the functional heterogeneity of attention layers. Methods like DuoAttention recognize that only a specific subset of attention heads—termed "Retrieval Heads"—require the full KV cache for long-context reasoning tasks [144, 149]. The vast majority of heads, termed "Streaming Heads," solely focus on local tokens and persistent attention sinks. By utilizing an optimization-based algorithm with synthetic data to identify retrieval heads, DuoAttention routes streaming heads to a constant-length cache and allocates full caches exclusively to retrieval heads. This context-aware dynamic routing achieves over 2.5x memory reductions and accelerates both pre-filling and decoding without accuracy degradation [144, 149].

This dynamic routing concept has been further extended by Flux Attention and the InfLLM family. Flux Attention utilizes a lightweight Layer Router to adaptively assign each layer to either exact FlashAttention or Sparse Attention based on the input query, translating theoretical computational reductions into practical wall-clock speedups [148]. InfLLM and InfLLM-V2 deploy dense-sparse switchable attention frameworks, where distant contexts are stored in block-level external memory units. The model uses dense attention for short inputs and smoothly transitions to sparse, block-level memory lookups for long sequences, reusing dense attention parameters through architecture modification to seamlessly adapt models from short to long sequences without catastrophic forgetting [150, 151, 154, 155].

Speculative Decoding and Generation Refinement

Autoregressive decoding is heavily bottlenecked by its sequential latency: the computation of token t_i strictly requires the completion of token t_{i-1} , leaving the massive parallel compute capabilities of modern GPUs severely underutilized. Speculative Decoding (SD) breaks this sequential dependency through a "draft-and-verify" generation-refinement framework, directly trading abundant parallel compute (FLOPs) for a reduction in memory-bandwidth-bound sequential operations [15, 16, 74].

In standard speculative decoding, a highly efficient, lightweight "draft model" autoregressively generates a sequence of candidate tokens γ [13, 14, 16]. Subsequently, the massive "target model" processes the entire drafted sequence in a single, parallel forward pass. A verification

algorithm evaluates the draft tokens against the target model's exact probability distribution [13, 92]. If a token is accepted, the system commits it to the output; if rejected, the target model's distribution is used to sample a corrective token, and the remaining drafted tokens are discarded [16, 35]. Because the verification algorithm mathematically guarantees that the final output distribution exactly matches the target model's distribution, SD provides lossless acceleration [13, 73, 118].

However, maintaining an external draft model introduces severe systems complexities, including the necessity of hosting two separate sets of weights, managing dual KV caches, and ensuring strict distributional alignment [74, 92]. To maximize the expected acceptance length, researchers have expanded SD from linear sequence drafting to tree-structured drafting. Because a draft model may output multiple high-probability candidates for a given position, constructing a token tree allows the target model to verify multiple potential trajectories simultaneously [16, 33, 73].

Frameworks such as Sequoia address the suboptimality of fixed, heuristic tree topologies by applying a dynamic programming algorithm to construct the optimal tree structure. Sequoia's tree construction algorithm ensures that the average number of generated tokens grows logarithmically with tree size, making it highly robust across different decoding temperatures and memory-bound regimes [156, 157, 161]. Similarly, OPT-Tree adapts the tree structure dynamically at each decoding step to maximize the mathematical expectation of the acceptance length [33, 158].

Verification traversal has also undergone significant refinement. Traditional top-down verification discards all child nodes the moment a parent node is rejected, wasting potentially valid subsequences deeper in the tree [34]. Algorithms like Traversal Verification invert this paradigm, utilizing a leaf-to-root traversal mechanism that considers sequence-level probability distributions, preserving valid subsequences and drastically increasing expected acceptance length while maintaining theoretical optimality [34, 35, 36].

A paradigm shift in SD occurred with the realization that token-level prediction suffers from high entropy. The EAGLE (Extrapolation Algorithm for Greater Language-model Efficiency) family of architectures revolutionized drafting by shifting speculation from the token space to the feature (hidden state) space [14, 115]. Instead of relying on an independent draft model, EAGLE appends a lightweight regression head directly to the target model [14, 115]. This head takes the

current hidden state v_i and predicts the subsequent hidden state v_{i+1} , which is then passed through the target model's original language modeling head to produce token probabilities [14]. By speculating in the feature space, EAGLE circumvents the requirement of learning the target model's exact vocabulary nuances, achieving significantly higher acceptance rates [14, 115]. EAGLE-2 further refines this by incorporating confidence-based dynamic draft trees, mitigating feature uncertainty as speculation extends deeper into the future [115, 118, 119].

Self-speculative methods bypass the need for external models entirely. Approaches like Medusa integrate multiple decoding heads into the base model, each trained to predict tokens at varying

future offsets ($t_{n+1}, t_{n+2}, \dots$) from the current hidden state, running all heads in parallel [109, 115]. Other frameworks achieve self-speculation by skipping intermediate transformer

layers during the drafting phase (e.g., SWIFT) or utilizing early exit strategies [99, 137]. Dynamic Depth Decoding and Delayed Tree Expansion delay the independent identically distributed (i.i.d.) branching points to preserve target distributions, matching context-dependent expansion decisions [36, 120].

Finally, Speculative Speculative Decoding (SSD) addresses the sequential dependency *between* the drafting and verification phases themselves. In standard SD, the drafter must wait for the target model to finish verification before starting the next draft. SSD frameworks, such as Saguario, parallelize these phases by predicting likely verification outcomes and pre-speculating draft trees asynchronously, eliminating drafting overhead entirely when predictions are correct [13, 31]. For multi-sample reasoning scenarios, consensus-driven draft extraction synthesizes high-quality draft tokens without auxiliary models by dynamically analyzing structural patterns across parallel reasoning paths via probabilistic aggregation [117].

SD Architecture	Drafting Mechanism	Verification Topology	Core Advantage
Standard SD [13, 16]	External lightweight LLM	Linear Sequence	Mathematical guarantee of lossless output
Sequoia [156, 161]	Dynamic programming optimized	Optimal Tree Structure	Scales expected acceptance logarithmically with tree size
EAGLE / EAGLE-2 [115, 118]	Lightweight feature regression head	Dynamic Draft Trees	Speculates in hidden state space, avoiding token entropy
Medusa [109, 115]	Multiple target decoding heads	Parallel Offset Prediction	Zero external models; self-contained prediction
Saguario (SSD) [13, 31]	Asynchronous pre-speculation	Parallelized Draft/Verify	Removes sequential dependency between draft and verify

Low-Precision Quantization Strategies

To directly combat the memory capacity and bandwidth limitations of LLM deployment, post-training quantization (PTQ) reduces the bit-width of model parameters, activations, and KV states. By utilizing low-bit integers (INT8, INT4, or sub-3-bit formats) or emerging microscaling formats (NVFP4), quantization shrinks the memory footprint and leverages high-throughput tensor cores [19, 23, 24, 176].

Weight and Activation Quantization

Weight-only quantization (e.g., W4A16) is highly effective at reducing parameter volume, yet it leaves the activation matrices in higher precision (FP16/BF16). While this alleviates weight-loading bandwidth, computations remain constrained by floating-point arithmetic [45, 78, 142]. Furthermore, quantizing weights can induce severe activation drift—mean shifts in post-attention LayerNorm outputs—necessitating frameworks like D2Quant, which utilize dual-scale quantizers to correct activation deviations dynamically [45, 78].

Achieving joint weight and activation quantization (e.g., W8A8 or W4A4) is notoriously difficult due to the presence of activation outliers—sparse but exceptionally high-magnitude values that dictate the dynamic range of the tensor [20, 87]. If the quantization scale is stretched to accommodate these outliers, the resolution for the majority of normal values is catastrophically degraded [20, 175]. SmoothQuant addresses this by leveraging a mathematically equivalent offline transformation that migrates the quantization difficulty from the activations to the weights [75, 76]. By smoothing activation channels with a predefined scaling factor and inversely scaling the corresponding weight channels, SmoothQuant enables near-lossless W8A8 execution without specialized kernels [75, 77].

Activation-aware Weight Quantization (AWQ) takes a different approach by observing that not all weights are equally critical. Using a small calibration dataset, AWQ identifies the top 1% of salient weights based on the magnitude of their corresponding activations [86, 88, 90]. By selectively protecting these salient weights through per-channel scaling optimization, AWQ drastically reduces quantization error without requiring mixed-precision hardware formats [88, 91]. Similarly, the GPTQ algorithm utilizes approximate second-order Hessian information to optimize weight rounding, allowing models to be aggressively compressed to 3-bit or 4-bit configurations in a single shot [81, 82, 83].

At extreme low-bit regimes (sub-3-bit), maintaining representation capacity requires isolating outliers entirely. SqueezeLLM introduced a Dense-and-Sparse quantization decomposition [170, 171]. It stores the vast majority of weights in an ultra-low precision dense matrix via sensitivity-based non-uniform quantization, while extracting the top 0.45% of critical outliers into an FP16 sparse matrix [171, 173]. During inference, the sparse and dense components are computed efficiently, circumventing the degradation typically associated with uniform ultra-low-bit rounding [171, 174].

Recent developments leverage orthogonal transformations to distribute outlier magnitudes. QuIP# utilizes the random Hadamard transform for incoherence processing. By multiplying weight matrices with random orthogonal matrices, QuIP# spreads outliers evenly across the

tensor, achieving state-of-the-art 2-bit quantization via highly symmetric E8 lattice-based vector quantization [176, 177]. Furthermore, extreme binarization techniques have emerged, such as W(1+1)A(1x4) configurations, where weights are quantized to 1-bit with an additional bit for fine-grained grouping, and activations are decomposed into Boolean variables, pushing the boundaries of LLM efficiency [46, 47].

KV Cache Quantization

Quantizing the KV cache presents unique challenges because it is generated dynamically at runtime, precluding the offline calibration utilized for model weights [71, 142]. Moreover, the key and value matrices exhibit divergent mathematical properties.

KIVI demonstrated that 2-bit KV cache quantization is viable through an asymmetric approach [68, 69, 70]. The analysis in KIVI reveals that key matrices possess a few fixed channels with massive magnitudes, necessitating per-channel quantization to confine errors to individual channels. Conversely, value matrices lack structured channel outliers but act as value mixers, making them suitable for per-token quantization [69, 70]. This asymmetric application drastically reduces peak memory usage without significantly harming perplexity [69].

However, aggressive uniform 2-bit compression can homogenize nuanced contextual dependencies, particularly in complex multi-step reasoning tasks. The Kitty framework addresses this through Dynamic Channel-wise Precision Boost, which ranks key cache channels by sensitivity. It maintains a small fraction of critical channels in higher precision (e.g., FP16) while compressing the vast majority to 2-bit, effectively maintaining long-context reasoning capabilities at near 2-bit aggregate memory costs [71].

The TriAxialKV framework exposes that KV quantization sensitivity varies across three orthogonal axes: temporal recency, modality, and semantic role [138]. By applying adaptive, multi-dimensional bit-width allocation, it avoids wasting bits on low-impact tokens and prevents over-compression of critical context [138]. Similarly, Adaptive KV relies on lightweight token-level features (entropy, attention variance, token frequency) to feed a data-driven controller that dynamically selects precision levels (2-bit, 4-bit, 8-bit, FP16) on the fly, functioning akin to a variable-length Huffman coding scheme for generation memory [42]. For VideoLLMs processing thousands of visual frames, VidKV introduces mixed-precision quantization in the channel dimension combined with Fast Fourier Transforms (FFT) for normal channels, compressing caches to 1.5-bit precision with virtually no performance drop [39].

Quantization Method	Target Tensor(s)	Core Optimization Strategy	Precision Target
SmoothQuant [75, 77]	Weights & Activations	Migrates activation outlier difficulty to weights	W8A8
AWQ [88, 91]	Weights	Protects 1% of salient weights based on	W4A16

		activations	
SqueezeLLM [170, 171]	Weights	Dense-and-sparse decomposition; non-uniform scaling	W3A16
QuIP# [176, 177]	Weights	Random Hadamard Transform; E8 lattice vector quant.	W2A16
KIVI [68, 69]	KV Cache	Asymmetric: Per-channel keys, per-token values	KV2

Algorithm-System Co-Design and Cross-Optimization

As individual optimizations reach their theoretical limits, the frontier of LLM serving relies on the deep cross-optimization of speculative decoding, KV cache manipulation, and quantization. Isolated algorithms frequently collide with production infrastructure realities. For example, speculative decoding's reliance on large candidate trees drastically increases the computational intensity of the verification phase, which can paradoxically erase the memory-bandwidth savings granted by 4-bit weight quantization [112, 116].

Quantized Speculative Decoding

To mitigate the verification bottleneck in memory-bound settings, frameworks like Quasar propose quantized self-speculative acceleration [92, 93]. Because verification is bounded by the memory bandwidth required to read the target model's weights, Quasar utilizes a heavily quantized variant of the target model specifically as the verifier [92, 93]. By shrinking the verification data movement, the system unlocks higher scaling limits for speculation lengths without hitting hardware ceilings.

Conversely, methods like QuantSpec tackle the drafting bottleneck in long-context scenarios [94, 95]. Small draft models historically suffer from poor acceptance rates over long sequences because they lack expansive context windows [95, 134]. QuantSpec employs the target model as its own drafter but executes the draft phase using a 4-bit hierarchical KV cache and 4-bit weights [95, 136]. The full-precision KV cache is reserved exclusively for the verification phase [136]. This enables the draft phase to maintain high alignment (>90% acceptance) due to shared architecture, while the low-bit cache bypasses the linear memory access penalty of long-context drafting [95, 135].

The QSpec paradigm completely decouples efficiency from quality by switching the target model between two precision modes at runtime: it utilizes low-precision joint quantization (W4A4) for high-speed drafting, and seamlessly switches to weight-only quantization (W4A16) for

high-fidelity verification [111, 113]. Because the base weights and the KV cache are structurally reused across both modes, the context-switch incurs near-zero overhead, circumventing the need for auxiliary models [113]. Similarly, SubSpec constructs highly aligned draft models by generating low-bit quantized substitute layers specifically from the offloaded portions of the target LLM, maximizing consumer GPU utility [96].

Hardware-Aware Attention Kernels and System Integration

Implementing W4A4 (4-bit weight, 4-bit activation) quantization on standard GPUs often yields theoretical memory savings but poor empirical throughput due to the severe runtime overhead (20-90%) of dequantizing partial sums on low-throughput CUDA cores [139, 140]. The QServe system resolves this via the QoQ (Quattuor-Octo-Quattuor: W4A8KV4) algorithm [139, 141]. QoQ utilizes progressive quantization—quantizing weights to 8-bit before converting to 4-bit—ensuring that all GEMM operations remain securely on the highly optimized INT8 Tensor Cores. By leveraging register-level parallelism and compute-aware weight reordering, QServe circumvents the sequential dequantization bottleneck, delivering measured end-to-end throughput improvements of up to 3.5x over standard TensorRT-LLM implementations [140, 143].

Attention kernels themselves are undergoing radical redesigns to exploit specific hardware asynchrony. FlashAttention-3 achieves up to 75% utilization of Hopper H100 maximum FLOPS (approaching 1.2 PFLOPs/s) by implementing warp-specialized software pipelining [182, 186]. It divides warps into dedicated "producers" (fetching memory asynchronously via the Tensor Memory Accelerator, TMA) and "consumers" (executing WGMMMA math instructions), successfully interleaving block-wise matrix multiplications and softmax operations to hide memory latency [182, 186, 187]. Furthermore, it employs incoherent processing via Hadamard transforms prior to FP8 quantization to spread outliers, reducing numerical error by 2.6x [182, 186].

When decoding long contexts, Flash-Decoding parallelizes the attention calculation across the keys/values sequence length rather than just the batch dimension [162, 165]. Because decoding typically processes a query length of 1, small batch sizes underutilize the Streaming Multiprocessors (SMs) on the GPU. Flash-Decoding splits the KV cache into chunks, computes attention across splits in parallel, and calculates log-sum-exp variables to reduce the results dynamically, maintaining exact attention while fully utilizing the GPU and yielding up to 8x speedups for 64k+ sequences [162, 165]. For workloads with heavy prefix sharing, FlashForge introduces a shared-prefix attention kernel that optimizes the memory hierarchy between shared and global memory, minimizing the redundant memory access bottleneck intrinsic to sequential decode stages [167]. Finally, vAttention provides verified sparse attention by combining

HashAttention with sampling theory, establishing user-specified (ϵ, δ) guarantees on approximation accuracy to ensure stable deployment of sparse attention at scale [122, 123].

Challenges and Prospects

While the amalgamation of cache compression, speculation, and quantization pushes serving throughput to unprecedented heights, it introduces nuanced challenges that must dictate future

research trajectories.

Alignment Collapse and Safety Vulnerabilities

A critical yet underexplored consequence of extreme KV cache quantization is "alignment collapse." Standard evaluations heavily index on perplexity and general task accuracy, which frequently appear stable under sub-4-bit compression. However, safety guardrails and alignment boundaries are disproportionately degraded [38, 67]. Safety and alignment features typically reside in low-dimensional activation subspaces. Because standard quantization parameters (like scaling factors) are dictated by high-magnitude outlier channels, the delicate low-dimensional safety signals are essentially crushed by quantization noise [38, 67]. Research demonstrates that models like Mistral-7B can lose over 15% of their refusal capabilities (generating unsafe content) at a negligible 1.03x perplexity increase under KV quantization [67]. Resolving this requires alignment-aware quantization diagnostics, such as Per-Channel Reduction (PCR), which identifies and protects overlapping safety channels from outlier-driven scale factors [38, 67].

Dynamic Policies over Static Heuristics

The field is actively transitioning from static heuristics to dynamic, learned policies. Rigid KV cache eviction methods that permanently delete tokens often trigger catastrophic failures in reasoning tasks, throwing models into repetitive loops if a critical semantic token is misidentified [25]. Furthermore, system prompt leakage remains a persistent threat in multi-instruction eviction, where rigid policies demonstrate eviction bias against specific instructions [130, 132]. Adaptive pipelines, such as PolyKV and ForesightKV, demonstrate that layer-wise heterogeneity and predictive modeling are necessary; earlier layers may require different cache budgets and eviction patterns compared to deeper layers, and eviction decisions should be guided by future attention score predictions formulated as Markov Decision Processes [107, 128]. Future serving engines will increasingly rely on integrated controllers—such as those seen in TurboSpec—that continuously monitor the "goodput" (rate of successfully verified tokens) and dynamically modulate speculation lengths, KV budgets, and batch sizes via closed-loop feedback, entirely removing manual tuning [114].

Hardware-Algorithm Symbiosis

Looking forward, the development of micro-scaling data formats (such as NVFP4 and MXINT) natively supported by newer hardware architectures (e.g., NVIDIA Blackwell) will redefine PTQ limits [48, 176]. By isolating outliers at the block level rather than the tensor level, micro-scaling formats prevent high-magnitude activations from inflating scaling factors across large matrices [176]. Systems like ARCQuant already demonstrate that bypassing mixed-precision hardware constraints by augmenting input tensors with quantized residuals can unlock W4A4 performance that rivals higher-precision bounds on unified data paths [176]. The continued symbiosis between customized CUDA/Triton kernels, speculative architectures, and low-precision data types will remain the central vector for scaling LLM deployment to edge devices and maximizing datacenter economics.

Conclusion

The deployment of Large Language Models has transcended the era of brute-force hardware scaling, entering a domain defined by rigorous algorithmic and systemic optimization. The memory bandwidth bottleneck, intrinsic to autoregressive decoding, has necessitated profound innovations across three primary axes. Key-Value cache optimization has evolved from naive contiguous allocation to advanced virtual memory paging, latent vector compression (such as Multi-Head Latent Attention), and theoretically grounded rate-distortion eviction. Speculative decoding has successfully broken the sequential barrier of text generation, advancing from simple token drafting to sophisticated feature-space prediction and optimal tree verification. Concurrently, post-training quantization has compressed weights, activations, and dynamic cache states into the sub-4-bit regime by strategically managing outlier distributions through dense-sparse decompositions and asymmetric channel scaling. Moving forward, the most significant gains in inference efficiency will not emerge from isolated algorithmic tweaks, but from holistic co-designs—where quantized draft models, dynamic verification graphs, and hardware-aware attention kernels operate synchronously to maximize both computational goodput and theoretical memory limits.

References

- [1] R. Tiwari, H. Xi, A. Tomar, C. Hooper, S. Kim, M. Horton, M. Najibi, M. W. Mahoney, K. Keutzer, and A. Gholami, "QuantSpec: Self-Speculative Decoding with Hierarchical Quantized KV Cache," arXiv preprint arXiv:2502.10424, 2025. [3] B. Li, Y. Jiang, V. Gadepally, and D. Tiwari, "LLM Inference Serving: Survey of Recent Advances and Opportunities," arXiv preprint arXiv:2407.12391, 2024. [4] P. Zeng, Z. Ning, J. Zhao, W. Cui, M. Xu, L. Guo, X. Chen, and Y. Shan, "The CAP Principle for LLM Serving: A Survey of Long-Context Large Language Model Serving," arXiv preprint arXiv:2405.11299, 2024. [7] Y. Xu, N. K. Khaira, and T. Singh, "KV Cache Optimization Strategies for Scalable and Efficient LLM Inference," arXiv preprint arXiv:2603.20397, 2026. [13] T. Kumar, T. Dao, and A. May, "Speculative Speculative Decoding," arXiv preprint arXiv:2603.03251, 2026. [15] H. Xia et al., "Unlocking Efficiency in Large Language Model Inference: A Comprehensive Survey of Speculative Decoding," arXiv preprint arXiv:2401.07851, 2024. [18] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, H. Jin, T. Chen, and Z. Jia, "Towards Efficient Generative Large Language Model Serving: A Survey from Algorithms to Systems," arXiv preprint arXiv:2312.15234, 2023. [20] Y. Chen, K. Zheng, Z. Guo, Q. Zhang, Y. Zhang, and J. Zhai, "A Survey of Quantization in LLM: Unlocking Potential Hardware Efficiency," *Journal of Computer Science and Technology*, vol. 41, no. 1, pp. 341-358, 2026. [25] Y. Lin, J. Ding, Y. Xing, P. He, J. Tang, and S. Mukherjee, "A Simple Plug-in for Improving Eviction-Based KV Cache Compression," arXiv preprint arXiv:2605.23258, 2026. [28] L. Moschella et al., "Learning to Evict from Key-Value Cache," arXiv preprint arXiv:2602.10238, 2026. [29] R. Miao, Z. Wang, W. Li, S. Wu, S. Liu, Y. Jiang, and T. Yang, "Beyond Token Eviction: Mixed-Dimension Budget Allocation for Efficient KV Cache Compression," arXiv preprint arXiv:2603.20616, 2026. [33] Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia, and B. Chen, "Sequoia: Scalable, Robust, and Hardware-aware Speculative Decoding," arXiv preprint arXiv:2402.12374, 2024. [34] R. Thomas, T. Kitanovski, M. Goldblum, and A. Pal, "Dynamic Delayed Tree Expansion For Improved Multi-Path Speculative Decoding," arXiv preprint arXiv:2602.16994, 2026. [38] B. C. Xu, A. Kumarappan, and M. Zhou, "Alignment

Collapse Under KV Cache Quantization: Diagnosis and Mitigation," arXiv preprint arXiv:2606.09864, 2026. [39] K. Tao, H. You, Y. Sui, C. Qin, and H. Wang, "Plug-and-Play 1.x-Bit KV Cache Quantization for Video Large Language Models," arXiv preprint arXiv:2503.16257, 2025. [42] S. P. H. Boroujeni, N. Mehrabi, P. Woods, G. Hillesheim, and A. Razi, "Don't Waste Bits! Adaptive KV-Cache Quantization for Lightweight On-Device LLMs," arXiv preprint arXiv:2604.04722, 2026. [46] S. Song, C. Wang, R. Wang, Y. Yang, and X. Zhang, "Achieving binary weight and activation for LLMs using Post-Training Quantization," arXiv preprint arXiv:2504.05352, 2025. [50] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," Proceedings of the Symposium on Operating Systems Principles (SOSP), 2023. [60] L. Jin et al., "SGLANG-LSM: A Log-Structured Merge-Tree based Storage Engine for Scalable LLM Serving," arXiv preprint arXiv:2511.16138, 2025. [61] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Re, C. Barrett, Z. Wang, and B. Chen, "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," NeurIPS, 2023. [68] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache," arXiv preprint arXiv:2402.02750, 2024. [71] T. Zhang, Z. Zeng, H. Peng, H. Zhuang, and C. Chen, "MixKVQ: Query-Aware Mixed-Precision KV Cache Quantization for Long-Context Reasoning," arXiv preprint arXiv:2512.19206, 2025. [75] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," ICML, 2023. [81] E. Frantar, S. Ashkboos, T. Hoeftler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," ICLR, 2023. [88] J. Lin et al., "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," MLSys, 2024. [103] Y. Feng, J. Lv, Y. Cao, X. Xie, and S. K. Zhou, "Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference," NeurIPS, 2025. [106] F. Liao et al., "DefensiveKV: Output Perturbation-Constrained KV Cache Selection," arXiv preprint arXiv:2502.03805, 2025. [114] A. Desai, "Full-Stack System Optimization for Speculative Decoding in LLMs," UC Berkeley EECS Technical Report, 2025. [118] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees," EMNLP, 2024. [121] R. Prabhu, A. Nayak, J. Mohan, R. Ramjee, and A. Panwar, "vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention," ASPLOS, 2025. [128] M. Oren, M. Hassid, N. Yarden, Y. Adi, and R. Schwartz, "Transformers are Multi-State RNNs (TOVA)," EMNLP, 2024. [139] Y. Lin et al., "QServe: W4A8KV4 Quantization and System Co-design for Efficient LLM Serving," arXiv preprint arXiv:2405.04532, 2024. [144] G. Xiao, J. Tang, J. Zuo, J. Guo, S. Yang, H. Tang, Y. Fu, and S. Han, "DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads," arXiv preprint arXiv:2410.10819, 2024. [148] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, "Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference," ICML, 2024. [151] C. Xiao et al., "InfLLM-V2: Dense-Sparse Switchable Attention for Seamless Short-to-Long Adaptation," arXiv preprint arXiv:2509.24663, 2025. [165] T. Dao et al., "Flash-Decoding for long-context inference," Stanford CRFM Blog, 2023. [171] S. Kim et al., "SqueezeLLM: Dense-and-Sparse Quantization," ICLR, 2024. [176] J. Chee, Y. Cai, V. Kuleshov, and C. De Sa, "QuIP: 2-Bit Quantization of Large Language Models With Guarantees," NeurIPS, 2023. [186] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and

Accurate Attention with Asynchrony and Low-precision," arXiv preprint arXiv:2407.08608, 2024.

[188] DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," arXiv preprint arXiv:2405.04434, 2024. [196] DeepSeek-AI, "DeepSeek-V3 Technical Report," arXiv preprint arXiv:2412.19437, 2024. [203] X. Fan et al., "MHA2MLA-VLM: Enabling DeepSeek's Economical Multi-Head Latent Attention across Vision-Language Models," arXiv preprint arXiv:2601.11464, 2026.